Лабораторная работа №2: Первоначальная настройка git

САКО ЛАССИНЕ

21.04.2026

Лабораторная работа №2: Первоначальная настройка git

Студент: САКО ЛАССИНЕ
Группа: НПИБД-02-25
Дата выполнения: 21.04.2026

Цель работы

Изучить идеологию и применение средств контроля версий.
Освоить умения по работе с git.

Ход выполнения работы

1. Настройка конфигурации git

Figure 1: Настройка git config

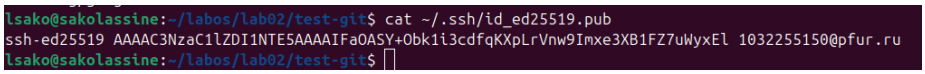

Figure 2: SSH ключ

2. Создание SSH ключа

3. Создание GPG ключа

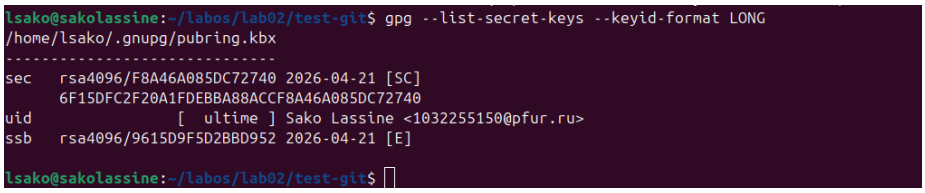

Figure 3: GPG список ключей

4. Проверка подписи коммита

5. GitHub с отметкой Verified

6. Отправка изменений на сервер

Выводы

В ходе выполнения лабораторной работы были получены следующие результаты:

1. Установлено программное обеспечение: git и gh.
2. Выполнена базовая настройка git (имя, email, autocrlf).
3. Созданы SSH ключи для безопасного подключения к GitHub.
4. Созданы GPG ключи для подписи коммитов.
5. Настроена автоматическая подпись коммитов.
6. Создан локальный и удалённый репозиторий.
7. Выполнен коммит с GPG подписью.
8. Проверена верификация подписи на GitHub.

Приобретённые навыки: - Работа с git в командной строке - Создание и управление SSH/GPG ключами - Работа с удалёнными репозиториями GitHub - Подписание коммитов для обеспечения подлинности

```
lsako@sakolassine:~/labos/lab02/test-git$ gpg --armor --export F8A46A085DC72740
-----BEGIN PGP PUBLIC KEY BLOCK-----

mQINBGnnn+MBEADV5hpUVBNPXKprJjLyqebxx+DSmWxopbpma4Lakd/jiwbSprsz
5Jii/xsvXx1T5OWgwRl0iyBeTu5oRdYYOvCLCF5D6sR6xt4KdeOWDkYVwGEAXbAm
KfemZJLxEF9M7mNuLWiOtYj4NznXaklfvMCY8xwoyxwZUxsyTkeGOCoFyTgX+NYl
eBOidcefrhutsEw3raE4zNKN4qFvoP2xlFDijWMnDh8laZHc3Y3RkxS73fMmn9wN
1v6VVzp7s8ywxEuwxhAI/ynA9kziSIpWK21aKQpqqHn8CS4oppAMgCih0ZvVuJtF
uxjS6v3nPhJKjgSYCKvHZ5oMJZPKL51VFE0PzvEsBOSPTpddbct9nyxQJc6xurpw
M2TpXZBaZxPrLfbNTNVsQ3GnCIdg6KkldSh5P+d0xOAwXTlMz9yQRcR7VKuGPfAu
mwRmmGNGtJqI7Vf6McsAnEq5PM60u8KaqQRr2RYAJxZ4n4KLK2lHNnnvsFYef36P
2BkB1Xl69tYLp8ACQ8k+1pgiiHMmOIh/xxi88jSLa8nBkEh/wn/fmihdlsW4cyNX
2fZWlFkKiJ2YAA8QdZpSOXU9tOrxijAQvdTBYeZRpebxfocNBwo8zUUuO+LW73xV
61kdgGVV2/kE7OApO9Xg0K7HnGMToLiRt0MAFXZdXiSsuaQFgSOevsReQwARAQAB
tCFTYWtvIExhc3NpbmUgPDEwMzIyNTUxNTBAcGZ1ci5ydT6JAlEEEwEKADsWIQRv
Fd/C8gof3ruoisz4pGoIXccnQAUCaeef4wIbAwULCQgHAgIiAgYVCgkICwIEFgID
AQIeBwIXgAAKCRD4pGoIXccnQDRGD/wK5BbKzzVBaDAq7NyjXS6e+4mUMzBAipha
sj9ZeaYpyQ9w7CD7IpuC1/6lfoLwenVkweWfDbl4jDcZ+pEdVkGoDCrDa6eFSoyI
acipEOFAFGiVVHkxRlR0VW+IzXWH+JVaKStoAv2LwewGTLMeTw0ClZ/62rM3OZPf
lSUnlRlVltMP6+Soo/3blaDxwU9/truS05jJp7XEfavzswRXykbyS8iV9lYfu9iK
quLxzGL4S9z0pvMNJsPoJo/o9fbiAdegQAot2yEyWWdEoIWtzX2OtXxyZDypE3Bv
8GTcWsiXunAsYomCDvxRE9ykY1Eqe0SS8qcTPmaydJuqOHHrX3bJRg1EdaEOpB9O
GoVMTLqMu0ugCzthTv+2c9x6SWBA3AX/qzGcT8CRuRXYCtF8imEIkZrfXG87xnYI
Pd7JmrVucMEf8Y/TWjZQwvKZNaZYdR3rhq7bR/3mqiep8IhuV5OX3KfZpqRGYZPf
GBkgARPr9L3b6FqVX/cXlTNc92VFSIZWLJpDYM6UPaJZQVkxokmv0ig/gsPCYato
qMBT580UABR5Hxp+Uio0ydHJBml7GFVjgfqrjnRbqBmJNuN92rkNFJIchnq5Qto+
mLGqucUMRynu5WtzJr321uHKXOoR1ONZYYT4KpRN5m1h9czdzSOFSOvW1h5bZwqQ
DZrowTRHG7kCDQRp55/jARAA090kpdQVGMOwtTtn/XoecuSdh9+llUU6AnThRkoI
oijLOzn7Msclw4K+2z8WG54tQ5D6HcCdXH9DN47ypF2n2dJ8QsPqtbxhci7M7qg5
icdhLZrzdytamEma4EASnnZr5aEvGxjmnq0Nzd0b4Li4Fl4R8B1axUxv+kEwxUXi
glDC1z/98iR05Cqmd8W027+uHXLkqTbAllmB4tuf5CypFk61VBiVf1w2vrdCRoxl
ej5u759BmeHWIO8Ni2qCImo00vJ5Mfd8qPmqlE22F6RINIyYDycwhe1ijkQlekdo
GmFgnu/4XxK2/YITX2OnLXVL0qZbndkPXfANy56YU1NHyJ1uDhP8gQYuioYZK4w9
e2PEWDQBMyybjO3NmXyM5Ef0j8+5tyeJ9W62mSAXhJI7x3rXmS4IqgleDsZylJ+e
H0bTXuvFeFEoLTZJ8kfvdjNYCvO5vEXjaNGEDF6bJ9IZIAqHQlnpSNnbN1QfNQXF
iu9OJ8iKg44+TsSL+OohAmLlAQGRqc92H8GgTO8p3Se1hdCpm/AfswMtHFOns82G
jAGB5NbDHzoDUKdKLuiu2VPHVHyxXewjqd5Gfrqew3SOwlHIUuxgztj4s1iKYMWA
FirIRbOkgdaUS55WJB2e9KwDqKZ4lOuZNKu1AXXBvsAioh8wX9i02NjceAAIMyiU
cjEAEQEAAYkCNgQYAQoAIBYhBG8V38LyCh/eu6iKzPikaghdxydABQJp55/jAhsM
AAoJEPikaghdxydAl1AP/1I0eFqzr+5bNf6OLX0/7fu3FDVrkSUNQLBOVScyY3Ce
VuCN/Gj5PiQp82s6NHPUp3MYc2AcLFrZz3JYEGnTXSob/E8LnrrlJE3Wuv4yjbki
W355N6g4J+7RrlxhGUuzzjKwXQg5aDs2cHav9YcJpiKbp+Xvir4cxepQkurI3M4I
gm0yFTa21x52L3Xjxig88JMf728v5UXv9pFrqX8JbUsK+8zp9YxGGAYgW69Olk+d
```

Figure 4: Экспорт GPG ключа

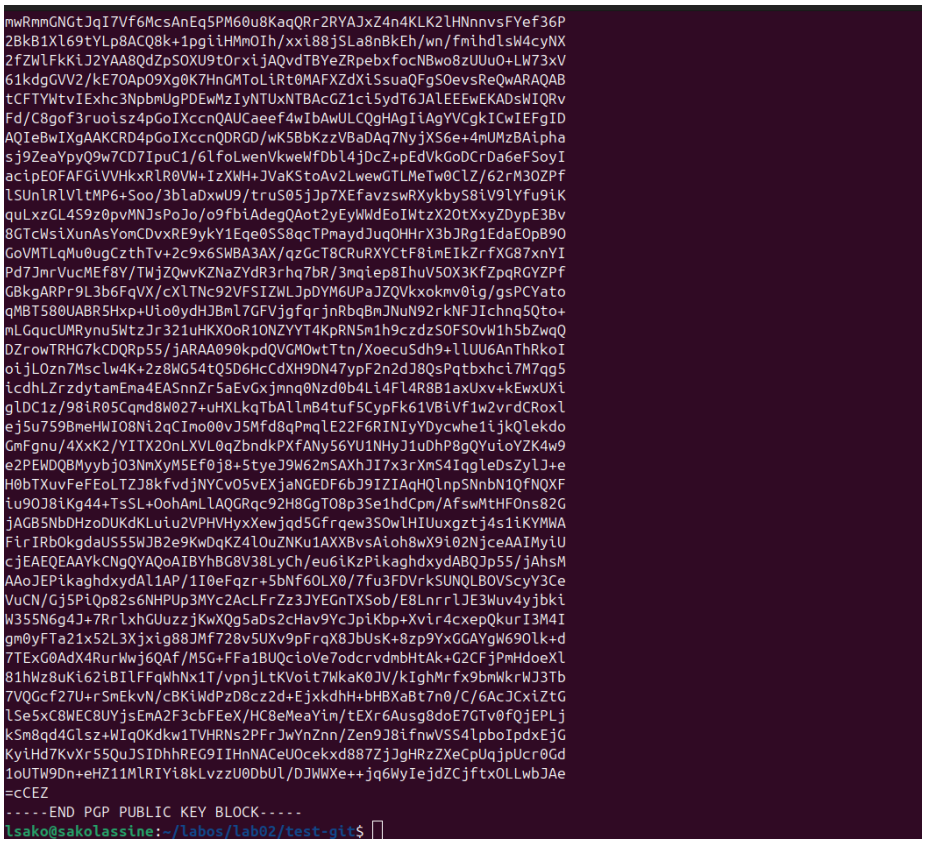

Figure 5: Проверка подписи

Figure 6: Проверка подписи

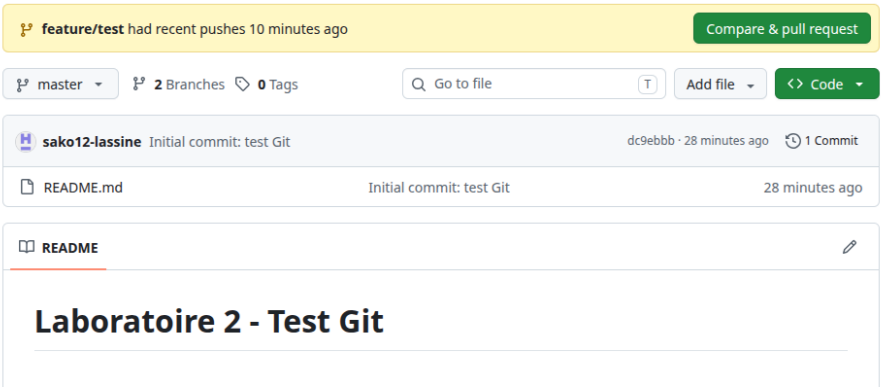

Figure 7: Verified на GitHub

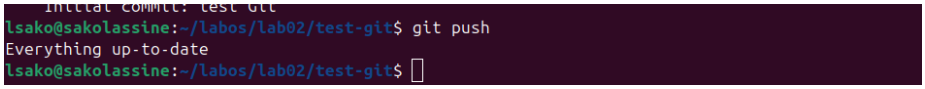

Figure 8: Git push

Ответы на контрольные вопросы

1. Что такое системы контроля версий (VCS) и для решения каких задач они предназначаются?

Системы контроля версий — это программные инструменты, отслеживающие изменения в файлах и каталогах проекта.

Задачи VCS: - Хранение истории изменений - Возможность отката к любой предыдущей версии - Совместная работа нескольких разработчиков - Разрешение конфликтов при слиянии изменений - Создание веток для параллельной разработки

2. Объясните понятия VCS: хранилище, commit, история, рабочая копия

Понятие	Описание
Хранилище (repository)	База данных, содержащая все версии файлов и историю изменений
Commit	Фиксация (сохранение) изменений в хранилище с комментарием
История	Последовательность всех коммитов, позволяющая отследить эволюцию проекта

Понятие	Описание
Рабочая копия	Локальная версия файлов, с которой работает пользователь

3. Централизованные и децентрализованные VCS. Примеры

Тип	Особенности	Примеры
Централизованные	Единый центральный сервер, клиенты получают последнюю версию	CVS, Subversion (SVN)
Децентрализованные	Каждый разработчик имеет полную копию репозитория	Git, Mercurial, Bazaar

4. Основные команды git

Команда	Назначение
`git init`	Инициализация нового репозитория
`git clone`	Клонирование удалённого репозитория
`git add`	Добавление файлов в индекс
`git commit`	Фиксация изменений
`git push`	Отправка изменений на сервер
`git pull`	Получение изменений с сервера
`git status`	Просмотр состояния рабочего каталога
`git log`	Просмотр истории коммитов
`git branch`	Управление ветками
`git merge`	Слияние веток

5. Что такое ветки (branches) и зачем они нужны?

Ветки — это независимые линии разработки, позволяющие вести параллельную работу.

Зачем нужны ветки: - Разработка новых функций без влияния на стабильную версию - Исправление ошибок в отдельных ветках - Эксперименты без риска сломать основной код - Параллельная работа нескольких разработчиков

6. Как игнорировать файлы при commit?

Создать файл `.gitignore` в корне репозитория и добавить шаблоны:

“‘bash *./log* .tmp **pycache**/ .vscode/

Зачем нужен .gitignore?

- **Исключить временные файлы** — файлы, создаваемые редакторами или компиляторами (например, `*.tmp`, `*.log`, `*.o`)
- **Не загружать конфиденциальные данные** — пароли, ключи API, файлы с секретами (например, `.env`, `config/secrets.yml`)
- **Уменьшить размер репозитория** — исключить большие бинарные файлы, кэш, зависимости (`node_modules/`, `__pycache__/`, `*.iso`)

Заключение

Лабораторная работа выполнена в полном объёме. Все цели достигнуты. Настроена базовая конфигурация git, созданы SSH и GPG ключи, выполнена интеграция с GitHub. Система готова к дальнейшей работе с системами контроля версий.